

Spring is not magic · part 2 of 2

What happens to your bean

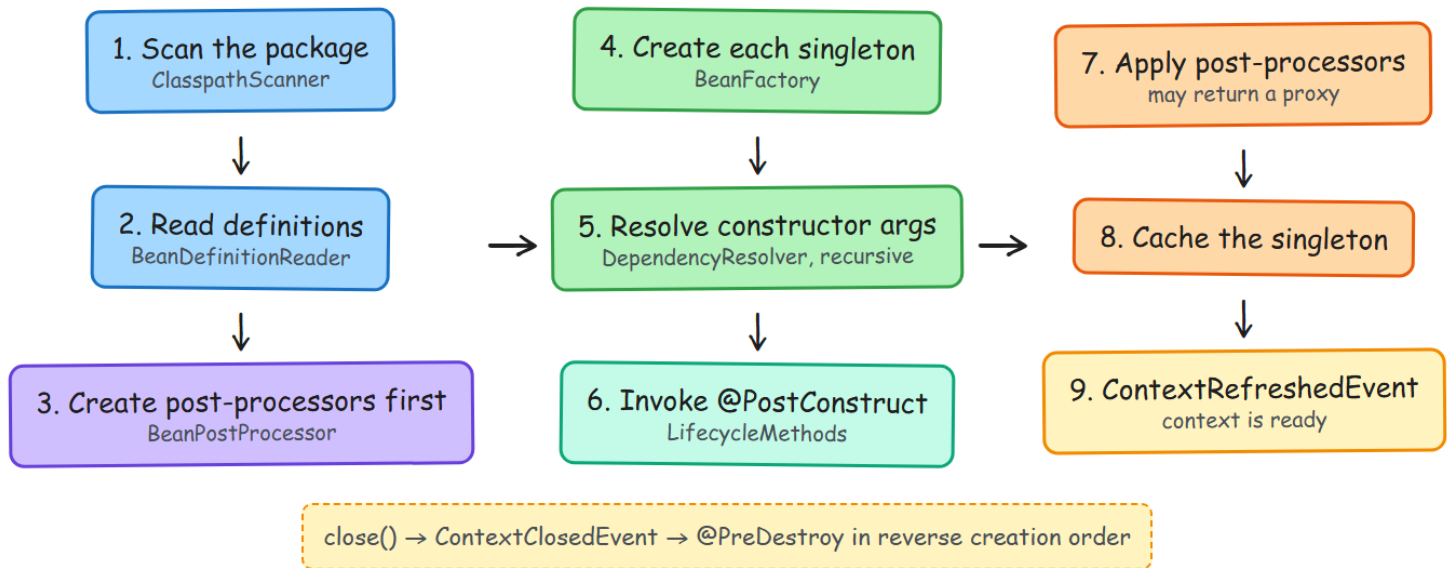
Post-processors, proxies, @Transactional, and how auto-configuration decides

github.com/ELMAALMIA/minispring

Ayoub EL Maalmi · Java & Spring Boot · 45 minutes

What happens when the context starts

`AnnotationApplicationContext.scan("com.example")`



Where we left off

Part 1, in four sentences

- An annotation does nothing. Something has to read it — and javap showed us exactly what it reads.
- A definition describes a bean; the factory creates it after its dependencies, recursively.
- Scopes decide how many exist; `@PostConstruct` and `@PreDestroy` decide when things happen.
- At that point the bean is built, initialised, and about to be handed to you.

Today: what happens between 'the bean is built' and 'you get it' — and who decides it should exist at all.



The seam

One method, and everything else is built on it

The seam: something may replace your bean

One method, and everything else follows

```
public interface BeanPostProcessor {  
  
    default Object postProcessAfterInitialization(  
        Object bean, String beanName) {  
        return bean;           // ...or something else entirely  
    }  
}
```

- The container creates the post-processors first, then passes every other bean through them
- A processor may return a different object — that one line is what makes AOP possible
- It runs after `@PostConstruct`, so a proxy always wraps a fully initialised bean

Everything in this talk is built on this single method.

Who reads what, part two

The annotations that need more than a definition

Annotation	Read by, in minispring	The Spring class	In XML, before
@Transactional	TransactionalProcessor	TransactionInterceptor	<tx:advice> + <aop:config>
@RestController	DispatcherServlet	RequestMappingHandlerMapping	<mvc:annotation-driven>
ApplicationListener	ApplicationEventMulticaster	SimpleApplicationEvent...caster	a <bean> implementing it
@Conditional	ConditionEvaluator	ConditionEvaluator (same name)	no equivalent

The last row is why Spring Boot could not have been built in the XML era.



2

Proxies

The container does not always hand you your bean

@Transactional is a proxy, and that explains the bug

TransactionalProcessor returns a different object than the bean



Inside the bean, "this" is the bean, not the proxy
so `this.placeOrder()` opens no transaction — the most common Spring bug

Same behaviour in Spring, with JDK proxies and with CGLIB subclasses

The bug you have already hit

The most common @Transactional mistake in production

```
@Override
public void placeAll(List<String> items) {
    items.forEach(this::placeOrder);    // "this" is the bean, not the proxy
}                                       // -> no transaction, and no warning
```

The test that pins it down:

```
void selfInvocationBypassesTheProxyAndStartsNoTransaction() { ... }
```

Same behaviour in Spring, with JDK proxies and with CGLIB. Fifteen lines of your own code explain a day of debugging.

Spring's other proxy: a generated subclass

Same idea, different trick — and it is what makes `@Configuration` work

```
OrderServiceImpl$$SpringCGLIB$$0 extends OrderServiceImpl
```

	JDK dynamic proxy (minispring)	CGLIB subclass (Spring Boot default)
How	implements the interfaces	extends the class, overrides its methods
Requires	an interface	a class and methods that are not final
<code>getBean(Impl.class)</code>	fails: the proxy is not that type	works: the proxy is a subclass
Built by	the JDK	bytecode generated at run time, with ASM
Self-invocation	bypassed	bypassed too — nothing changes

It also rewrites `@Configuration` classes, so one `@Bean` method calling another returns the singleton. That is the one thing minispring cannot do.



3

Invent your own annotation

The moment this stops being someone else's framework

@Timed, in three steps

The same pattern as @Transactional

```
@Retention(RUNTIME) @Target(METHOD)
public @interface Timed { }
```

1. The sticker

```
public Object postProcessAfterInitialization(Object bean, String name) {
    // if a method carries @Timed, return a proxy that measures the call
    return Proxy.newProxyInstance(loader, interfaces, handler);
}
```

2. A BeanPostProcessor that wraps the bean · 3. A test that proves the duration is logged

Thirty lines. You have just written the mechanism behind @Cacheable, @Retry and @Async.



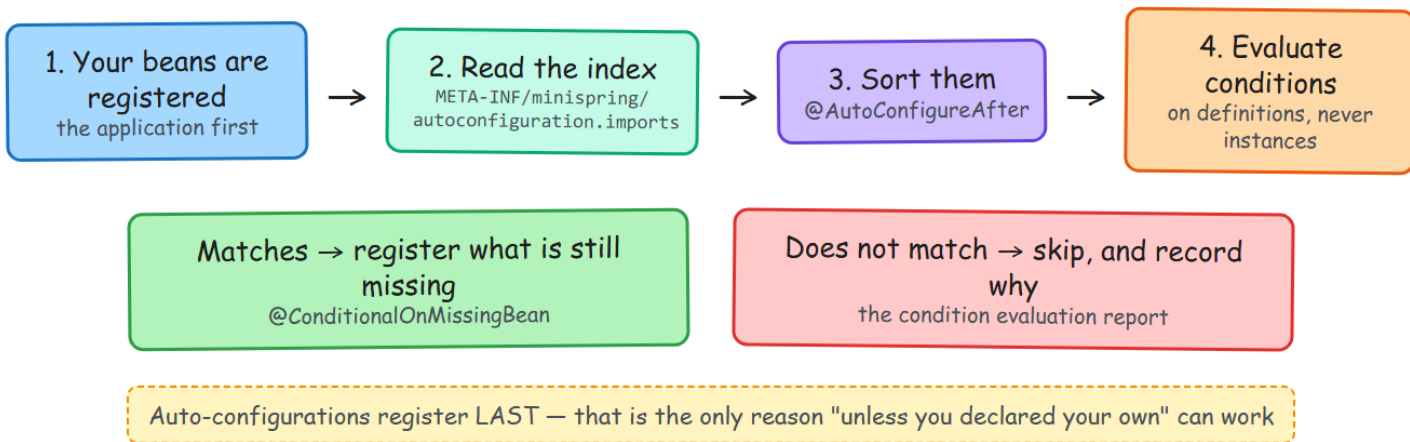
4

Auto-configuration

How @Conditional decides what gets wired

How @Conditional decides what gets wired

Auto-configuration is a plugin on the container, not a power of it



Two rules, and everything follows

1. Conditions read definitions, never instances

Deciding whether a bean should exist must not create beans, or the question answers itself.

2. Auto-configurations register last

Only then can `@ConditionalOnMissingBean` mean "unless you declared your own".

```
@Bean
@ConditionalOnMissingBean
public Notifier notifier() { return message -> System.out.println("NOTIFY " + message); }
```

Same classes, opposite registration order, different application. There is a test for it.

The @ConditionalOnX family

Each one is an ordinary @Conditional with a name

Annotation	Registers the bean when	What it is for
@ConditionalOnClass(name=)	every named class is on the classpath	a starter stays quiet without its library
@ConditionalOnMissingClass	none of them is	fall back to another implementation
@ConditionalOnBean(X.class)	a bean of that type is already there	complete what the application started
@ConditionalOnMissingBean	no bean of that type exists yet	provide the default, and step aside
@ConditionalOnProperty	a property says so	let people switch you off

The container knows none of them. Writing your own condition takes about a dozen lines.

"Why is my bean not there?"

Every decision is recorded, with its reason

```
CONDITION EVALUATION REPORT
```

```
Positive matches:
```

```
none
```

```
Negative matches:
```

```
NotifierAutoConfiguration
```

```
- property 'minispring.notifier.enabled' is 'false', expected 'true'
```

In Spring Boot: start with `--debug`. In minispring: `minispring.autoconfigure.report=true`.



5

What it costs

The honest slide

100 ms against 1.7 s, and why that is fair

The same application, on both containers

Spring Boot does this	minispring does not
Evaluates hundreds of conditional auto-configurations	reads one small index file
Reads annotation metadata from the whole classpath with ASM	reflects over one package
Builds an Environment from many property sources	four sources, in order
Initialises a logging system, generates CGLIB subclasses	neither
Runs a real servlet container	a 100-line HTTP dispatcher

The gap is not a flaw in Spring. It is the price of the features I chose not to build.

The whole thing, in numbers

So you know what 'small' means here

1,500

lines, both modules

0

runtime dependencies

129

tests, no mocks

92%

coverage, gate at 90%

Small enough that every claim in this talk can be checked in one afternoon.

Three things to take home

1

A post-processor may hand you a different object

That single method is where `@Transactional`, `@Cacheable` and `@Async` come from.

2

Self-invocation bypasses the proxy

Inside the bean, this is the bean. No proxy, no transaction, no warning.

3

Conditions read definitions, and run last

That is the only reason "unless you declared your own" can work.

github.com/ELMAALMIA/minispring

Read it commit by commit: one concept each, all green.